

b) Investigación Seguridad:

El equipo investiga la manera de aplicar seguridad en sus aplicaciones.

➤ El equipo investiga como se debe aplicar conceptos de seguridad en las distintas etapas del desarrollo de software en un ciclo de vida, detalla mediante un cuadro cuales son las buenas prácticas que se deben tener en cuenta. Dar ejemplos de aplicaciones inseguras que no utilizan seguridad en el desarrollo de software. Por Ejemplo: <https://www.instagram.com/reels/DPNJZL2ESk0/>

Etapas 1 — Requerimientos / Análisis

- Definir requerimientos de seguridad a medida que se definen los requerimientos funcionales: ¿qué datos son sensibles? ¿Qué roles existen y qué puede hacer cada uno?
- Identificar normativas aplicables.

Etapas 2 — Diseño

- Modelado de amenazas: por cada funcionalidad, preguntarse "¿Cómo podrían vulnerarla?".
- Aplicar principios de diseño seguro:
 - **Menor privilegio:** cada usuario/rol solo puede hacer lo mínimo necesario.
 - **Defensa en profundidad:** no confiar en una sola barrera.
 - **Fail securely:** si algo falla el sistema rechaza de manera automática
 - **Separación de responsabilidades:** separar la parte que autentica de la que autoriza.

Etapas 3 — Desarrollo / Implementación

- No almacenar contraseñas en texto plano
- Validar toda entrada del usuario
- Usar queries parametrizadas
- No dejar datos sensibles en el código
- Revisión de código entre pares
- Análisis estático de código

Etapas 4 — Testing

- Tests de seguridad específicos, no solo funcionales
- Análisis dinámico mientras la aplicación esté corriendo

- Escanear vulnerabilidades y exposiciones comunes en las dependencias
- Pentesting solo si es necesario

Etapas 5 — Despliegue

- Configuración segura del servidor
- HTTPS obligatorio
- Gestión de contraseñas de la base de datos o configuraciones de la aplicación fuera del código.
- Separación de ambientes (dev/staging/prod) con credenciales distintas en cada uno.

Etapas 6 — Mantenimiento / Operación

- Logging y monitoreo.
- Actualización de dependencias.

Categoría	Buena práctica
Autenticación	No guardar contraseñas en texto plano Limitar intentos de login
Autorización	Principio de menor privilegio Defensa en más de una capa Rechazar por defecto
Datos	Consultas parametrizadas Escapar (encode) la salida HTML Token anti-CSRF
Transporte	Cifrar la comunicación
Configuración	No dejar información sensible en el código Deshabilitar detalles de error en producción
Validación	Validar toda entrada del usuario
Dependencias	Mantener librerías actualizadas
Testing	Test de seguridad además de funcionalidades Análisis estático y dinámico
Operación	Logging sin datos sensibles

➤ ¿Qué significa que una clave de usuario se encuentre encriptada? Explicar y desarrollar los siguiente ejemplo el método de encriptación explicado en el siguiente video <https://www.youtube.com/watch?v=7-muJODqU6g>

Que la clave del usuario esté encriptada significa que sufrió algún tipo de proceso matemático en donde dejó de ser texto plano y se transformó en texto ilegible. De esta forma si un atacante intercepta la comunicación o accede a la base de datos no puede ver la contraseña en texto claro de manera directa.

```
public class EncryptUtil {

    public static void main(String[] args) {
        String password = "123";
        System.out.println("Contraseña original: " + password);
        String encryptedPassword = encryptPassword(password);
        System.out.println("Contraseña encriptada: " + encryptedPassword);
    }

    public static String encryptPassword(String password) {
        BCryptPasswordEncoder passwordEncoder = new BCryptPasswordEncoder();
        return passwordEncoder.encode(password);
    }
}
```

Este código hashea una contraseña usando BCrypt, un algoritmo de una sola vía. El resultado tiene el formato:

\$2a\$10\$N9qo8uLOickgx2ZMRZoMyelJZAgcfl7p92ldGxad68LJZdL17lhWy,

compuesto por: la versión del algoritmo (2a), el cost factor (10, que indica $2^{10} = 1024$ iteraciones internas, pensado para hacer lentos los ataques de fuerza bruta), el salt (22 caracteres, un valor aleatorio e independiente generado en cada llamada a encode()), y el hash resultante (31 caracteres, producto de aplicar el algoritmo a la contraseña + el salt).

Para verificar un login, simplemente se extrae el salt del hash guardado, se vuelve a calcular el hash de la contraseña que el usuario ingresó usando ese mismo salt, y se comparan los dos hashes completos. Si coinciden, la contraseña es correcta.